Project 7 (Bonus) — MCPForge-Local-Notes-And-Tasks-Server

Title: MCPForge Local Notes And Tasks Server

Type: MCP Server (2 Tools, 1 Resource, 1 Prompt) + a client · **Stack:** Python, official `mcp` SDK (MCPServer), SQLite, MCP Inspector · **Level:** Intermediate (bonus / portfolio project)

Tagline: A real MCP server — Local Notes & Tasks — exposing 2 Tools, 1 Resource, and 1 Prompt over the Model Context Protocol, usable by any MCP-compatible client without a custom integration for each one.

> This is the **reading guide** — the story, the folder structure and why each part exists, setup, the 4 build steps, and which document to read for each idea. The other file, README.md, is the **repo README** — you copy it into your repo folder at the end and push it to GitHub. You write every folder, file and line of code yourself; the only thing already created for you is the empty repo folder `MCPForge-Local-Notes-And-Tasks-Server/` .

Charter (what this project is)

A real MCP (Model Context Protocol) server, built with the official `mcp` Python SDK, that exposes 2 Tools, 1 Resource, and 1 Prompt over stdio — something any MCP-compatible client (Claude Desktop, your own agent code, an IDE) can connect to and use, without you writing a custom integration for each one. This project proves two things with real code: (1) you understand the actual mechanics of an MCP server — Tools, Resources, Prompts, and the client/server handshake underneath them — and (2) you can tell, for a real piece of data or behavior, which of the three it actually belongs on. There is **no LLM call anywhere** inside the server — this is about the protocol, not about calling a model.

Jump to: Read first · Project structure · Setup · How testing works · Step 1 · Step 2 · Step 3 · Step 4 · Run and test · Checklist · Publish

The Story — what you're actually building

Imagine you want the same "notes and tasks" capability available not just in one app, but in Claude Desktop, in your own agent code, and in an IDE plugin — three completely different programs, none of which know anything about each other or your database. Without MCP, you'd write three separate

integrations — the same add-a-note logic, copy-pasted and adapted three times, drifting out of sync the moment one copy changes.

MCPForge is the alternative: you build the notes/tasks logic exactly once, as an MCP **server**, and expose it over the protocol. Any MCP-compatible **client** can connect to that one server and discover what it offers, at runtime, without a single line of client-specific integration code. The server is a small but real "Local Notes & Tasks" tool: `add_note` and `search_notes` as Tools (they *do* something — write or search), `notes://all` as a Resource (read-only data, nothing to act on), and `summarize_notes` as a Prompt (a reusable template, not a hardcoded string buried in one app). Getting each of these three right — not just cramming everything into Tools because that's the one you understand best — is the actual point.

Each step builds one piece, in order: Step 1 proves the connection itself works, with the smallest possible server. Step 2 makes the data real — a SQLite database that survives a restart, with input validated instead of silently accepted. Step 3 adds the Resource and the Prompt, and forces you to think about *why* each isn't just a third and fourth Tool. Step 4 builds the client side — a script that connects, discovers, and uses everything, proving the full loop works end to end.

> **Before you read further — think about it yourself:** `add_note` clearly needs to be a Tool. But `notes://all`, a plain listing of every note, *could* be built as a third Tool — what would you lose? And `summarize_notes` could be a prompt string you type into Claude Desktop by hand — what does having the server provide it actually buy you? Sit with these before you read the Steps.

What you're building, in one line: a small MCP server exposing notes-and-tasks tools, a resource, and a prompt over one standard protocol, plus a client that uses them.

Why this needs to exist: without a shared protocol, every app that wants your notes/tasks logic needs its own hand-written integration — and separate copies drift apart the moment one changes.

When you'd reach for this at a real job: when more than one program — Claude Desktop, an IDE plugin, your own agent code — needs the same capability, and you don't want to build and update a separate integration for each.

How it works, mechanically: the server declares its Tools, Resources, and Prompts with plain decorators; any MCP client connects over stdio, calls `list_tools()` / `list_resources()` / `list_prompts()` to see what's on offer, then calls whichever it needs by name.

Why not just write a Python function the agent calls directly? Simpler and faster — as long as it's one agent, in one codebase, that you control. It stops working the moment a second client (a different language, a different team, Claude Desktop instead of your script) wants the same capability: now you're copy-pasting the logic into every consumer, and a bug fix in one copy never reaches the others. MCP's value is letting different clients and languages share one implementation — it's not a speed win; a direct call is still faster, it's just not shared.

Where This Fits

Projects 1-6 are one continuous arc — the same core skills applied to a growing number of agents. This bonus project is not part of that arc. It doesn't add agents, and it isn't built on LangChain or LangGraph — there's no LLM call anywhere inside the server. What it teaches instead is one specific, widely-used technology: **MCP server-building**. Docs 01-06 already taught the underlying ideas MCP standardizes — a tool's precise description, its typed argument shape, clean error handling instead of a crash (all Doc06), and config/logging discipline (Doc01). MCP, covered in Doc06's later sections, is the protocol that lets you expose those same ideas *once*, over a standard connection, instead of re-wiring them into every app. Build it because MCP is genuinely used right now — Claude Desktop, Cursor and other IDEs all speak it — and "I built and tested a real MCP server" is a specific, checkable portfolio line.

What to Read First (docs this project uses)

Everything this project needs was taught in Docs 01 and 06:

You need to understand…	Read this	Used in
venv, `.env`, never committing secrets	Doc01 — `.env` files	Setup
`load_config()`, `get_logger()`	Doc01 Build Task	Step 1
What MCP is, and the problem it solves	Doc06 — Core Concepts	Charter, Step 1
MCP's 3 building blocks: Tools, Resources, Prompts	Doc06 — Core Concepts	Step 3
A tool's description is really a prompt	Doc06 — Core Concepts	Steps 1, 3
Getting a tool's failure back correctly	Doc06 — Core Concepts	Step 2
MCP servers and clients, how they connect	Doc06 — Core Concepts	Steps 1, 4
Reading a value safely (fail loudly)	Doc01 — Core Concepts	Step 2

Reused code, unchanged except for its folder and import lines: `config.py`, `exceptions.py`, `utils/logger.py` (Doc01). The only Doc01 twist is that the logger writes to **stderr**, because on the stdio transport stdout is the protocol channel.

Project Structure (the professional layout)

Same rule as before: only folders with a real job. The code lives in a `src/` package; the two entry points — `server.py` (the server) and `client.py` (the client) — sit at the repo root, because that's what an MCP server's and client's entry files are: `mcp dev server.py` runs the server, `python client.py` runs the client.

Names: the repo folder vs. the Python package

- **Repo folder:** `MCPForge-Local-Notes-And-Tasks-Server/` — already created, and the exact GitHub repo name.
- **Python package:** `src/`, right next to `server.py` and `client.py`. The shared code imports from it the same way: `from src.db import insert_note`.
- **Files:** `snake_case.py`, one job each; test files start with `test_`. The server is `server.py`, the client is `client.py`.
- **Resource URIs:** `notes://all` — a scheme (`notes`) plus a path, the MCP way of naming a resource.

The final layout (after Step 4)

```
<repo folder>/   (MCPForge-Local-Notes-And-Tasks-Server/)
├── src/
│   ├── utils/
│   │   ├── __init__.py
│   │   └── logger.py          get_logger() -> stderr       (Doc01)
│   ├── __init__.py
│   ├── config.py              Config, load_config()        (Doc01)
│   ├── db.py                  init_db/insert/find/list_all
│   └── exceptions.py          MissingConfigError, EmptyNoteError
├── tests/
│   ├── integration/
│   │   └── test_server.py     drives the server via a client
│   └── unit/
│       ├── test_config.py     no server needed
│       └── test_db.py         no server needed
├── .env                       LOG_LEVEL, NOTES_DB_PATH — never committed
├── .env.example               the setting names, no secrets
├── .gitignore                 keeps .env, .venv/, notes.db out of git
├── client.py                  the standalone MCP client (Step 4)
├── LICENSE                    lets others reuse your code (MIT)
├── README.md                  the repo's front page (copied in at end)
├── requirements.txt           the libraries, for pip install -r
└── server.py                  the MCP server: 2 Tools, 1 Resource, 1 Prompt
```

`notes.db` (the SQLite file, created by Step 2) is in `.gitignore` — it's generated locally, not committed.

Why each folder exists

- **`src/`** — the package: config, errors, the SQLite helpers, the logger. **Why:** `server.py` and `client.py` are entry points; the logic they share lives here, testable on its own.

- **`src/utils/`** — the logger. **Why:** it's used by the server, and it writes to stderr (not stdout) for a protocol reason specific to this project.
- **`tests/unit/`** — the config and the SQLite helpers, with no server. **Why:** fast, free, no process to launch.
- **`tests/integration/`** — the server, driven by a real MCP client over stdio. **Why:** an MCP server can only be exercised through a client; this proves the exact code path a real one uses.

Why each root file exists

- `.env` — `LOG_LEVEL` and `NOTES_DB_PATH`; never committed. Neither is a secret (this server has no API key), but keeping local paths out of git is still the habit.
- `.env.example` — which settings to create, with placeholder values.
- `.gitignore` — keeps `.env`, the venv, cache files and `notes.db` out of git.
- `client.py` — the standalone MCP client: run `python client.py`.
- `LICENSE` — the MIT license text, created in Setup.
- `README.md` — the repo's front page on GitHub.
- `requirements.txt` — the library list for `pip install -r requirements.txt`.
- `server.py` — the MCP server: run `mcp dev server.py`, or let `client.py` launch it.

Why each file exists (what and why)

Every file you'll write, in one place. The steps below build these one at a time; come back here whenever you forget why a file exists.

- `src/utils/logger.py`
  - **What:** One function, `get_logger(name)`, that adds a console handler on **`sys.stderr`** the first time, with a timestamp format and the level from `LOG_LEVEL`.
  - **Why:** Every file logs the same way — and on stderr, never stdout, because on the stdio transport stdout *is* the MCP protocol channel.
- `src/config.py`
  - **What:** A `Config` dataclass (`log_level`, `notes_db_path`), `require_env(key)` that raises `MissingConfigError` when a setting is empty, and `load_config()`, which reads both settings (both optional, with defaults) and returns a `Config`.
  - **Why:** Every file gets its settings from one place. This server has no API key, so nothing here is required — but the pattern is the same as every other project.
- `src/db.py`
  - **What:** `init_db(path)` (creates the `notes` table), `insert_note(path, text)` (returns the new id from `lastrowid`), `find_notes(path, query)` (a case-insensitive substring search), and `list_all_notes(path)` (every row, oldest first).
  - **Why:** All SQL behind small named functions, testable with no server — so a search bug is caught here, not mistaken for a model mistake.

- `src/exceptions.py`
  - **What:** `MissingConfigError` and `EmptyNoteError`, each a small `class Name(Exception): pass` with a docstring.
  - **Why:** Code can catch one specific problem. `EmptyNoteError` names the one failure `add_note` guards against.
- `__init__.py` (every code folder)
  - **What:** An empty file in `src/` and `src/utils/`.
  - **Why:** Python only treats a folder as a package when this file is there, so `from src.db import ...` resolves.
- `server.py`
  - **What:** Builds `mcp = MCPServer("notes-server")`, declares `add_note` and `search_notes` as `@mcp.tool()`, `notes://all` as a `@mcp.resource(...)`, and `summarize_notes` as a `@mcp.prompt()`, then `mcp.run(transport="stdio")`. `add_note` validates its input and returns a clear error string on empty text.
  - **Why:** The one file that *is* the server. Everything a client can discover and call is declared here.
- `client.py`
  - **What:** Launches `server.py` as a subprocess over stdio, calls `list_tools/list_resources/list_prompts`, then uses all three building blocks (`call_tool` twice, `read_resource`, `get_prompt`), wrapped so a connection failure prints one clear line.
  - **Why:** Proves the full connect → discover → use loop works from the outside — the exact shape a real agent's MCP integration uses.
- `tests/unit/test_config.py`
  - **What:** Three cases, no server, each printing `OK` or `WRONG`: `require_env()` raises for a missing and an empty made-up setting, and `load_config()` returns a `Config` with the default db path.
  - **Why:** Proves the settings code works with no process to launch.
- `tests/unit/test_db.py`
  - **What:** Four cases on a throwaway database, each `OK` / `WRONG`: insert returns id 1, a case-insensitive search finds a note, a no-match returns `[]`, and `list_all_notes` returns the rows in order.
  - **Why:** Catches an SQL bug — a case-sensitive search, a lost row — for free, no server.
- `tests/integration/test_server.py`
  - **What:** Launches `server.py` with a real client and checks six things, each `OK` / `WRONG`: the tools are listed, `add_note` works, search is case-insensitive, an empty note is rejected, the Resource lists notes, and the Prompt is a filled-in template.
  - **Why:** The only way to exercise an MCP server is through a client — this proves the whole protocol surface end to end.

How imports work in this layout

- Every folder with Python files gets an empty `__init__.py` (`src/`, `src/utils/`). It marks the folder as a package, so `src.db` is importable.
- `server.py` and `client.py` sit beside `src/` at the repo root, so `from src.db import ...` resolves when you run from the repo root.
- `client.py` launches the server with `StdioServerParameters(command="python", args=["server.py"])` — because its working directory is the repo root, the subprocess finds `src/` there too.
- Tests run as modules from the repo root: `python -m tests.unit.test_db`, `python -m tests.integration.test_server`.
- Always run commands from the **repo root** — `notes.db` and `src/` are found from there.

Left out on purpose (and when they come)

Folder / file in a big template	Why not here	Where it appears
An LLM call anywhere	This is a protocol server, not an agent	Every other project
`pyproject.toml`, `pip install -e .`	A flat `src/` run from the repo root is enough	—
An HTTP transport	stdio is the simplest real transport, and what Claude Desktop uses	When a server must be reachable over the network
Auth / API keys for callers	A local, personal server	A hosted MCP server
A second Resource or Prompt	One of each is enough to learn the distinction	Project 11 (MCPCrew) and beyond
pytest	Plain OK/WRONG scripts, same as Projects 1-6	Doc13

How the structure grows, step by step

Step	New files	Changed files
Setup	root files, `src/__init__.py`	—
1	`config.py`, `exceptions.py`, `utils/logger.py`, `server.py`, `tests/unit/test_config.py`, `tests/integration/test_server.py`	—
2	`db.py`, `tests/unit/test_db.py`	`exceptions.py`, `server.py`, `test_server.py`

Step	New files	Changed files
3	—	`db.py`, `server.py`, `test_db.py`, `test_server.py`
4	`client.py`	—

Setup (do this once, before Step 1)

Read first: Doc01 — Virtual environments, Doc01 — `.env` files.

1. Go into the repo folder, start git, and make a venv. From the curriculum's root folder (`multi-agent-ai-engineering-curriculum/`):

```
cd project_7_mcpforge_mcp_server
cd MCPForge-Local-Notes-And-Tasks-Server
git init
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
```

2. Create `.gitignore` **before** your first commit:

```
# secrets / local settings — never pushed
.env
# the virtual environment — rebuilt from requirements.txt
.venv/
# Python's compiled cache files
__pycache__/
*.pyc
# the local notes database (Step 2) — generated, not committed
notes.db
```

3. Create `.env` and `.env.example`. Neither key is a secret (this server never calls an LLM), so real-looking values are fine in both:

```
# .env — never commit this file
LOG_LEVEL=INFO
NOTES_DB_PATH=notes.db
```

```
# .env.example — commit this one
LOG_LEVEL=INFO
```

```
NOTES_DB_PATH=notes.db
```

4. Create `requirements.txt`:

```
mcp[cli]>=2.0
python-dotenv>=1.0
```

`mcp[cli]` installs the SDK **and** the **MCP Inspector** — a small local web UI that connects to your server, lists its Tools/Resources/Prompts, and lets you call them by hand. Run it with `mcp dev server.py` — the fastest way to test a server before writing any client code.

5. Create an empty `src/__init__.py`, then install:

```
pip install -r requirements.txt
```

Check it worked: from the repo folder, `python -c "import src; print('ok')"` prints `ok`.

6. Create `LICENSE` — the MIT license, so others may reuse your code. Put the current year and your own name on the Copyright line:

```
MIT License

Copyright (c) 2026 Your Name

Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
SOFTWARE.
```

A Real Example (so this isn't just theory)

Scenario: a small local notes tool. Each note is free text plus when it was created.

Test these 3 inputs for `add_note`: 1. `"Buy milk and eggs"` → succeeds, returns `"Note added with id 1."` 2. `"Call the dentist to reschedule"` → succeeds, returns `"Note added with id 2."` 3. `""` or `" "` (only whitespace) → **rejected** with a clear error, never stored as a blank note.

Then: - `search_notes("dentist")` → returns `["Call the dentist to reschedule"]` — note 1 doesn't match, and note 3 was never stored. - Reading `notes://all` → a read-only listing of both real notes, with their ids. - Using `summarize_notes` with a date range → a filled-in prompt *template* asking a model to summarize the notes created in that range. The server produces the prompt text, not a summary — whichever client you're using sends that prompt to a model.

Why input 3 matters: it's the same "don't let bad input slide through" lesson Project 1's missing-field case taught — here it's an empty *required* field, and the fix takes the same instinct: catch it on purpose, fail with a clear message, never store something you already know is invalid.

Why This Project Is Good for Your Portfolio

- **What problem it solves:** every AI app that wants your notes/tasks logic currently needs its own custom integration. This builds that logic once, behind a standard protocol, so any MCP-compatible client can use it without a rewrite.
- **Why it matters:** MCP is a real, currently-used standard — Claude Desktop, Cursor and other tools speak it. "I built and tested a real MCP server" is a specific, checkable claim an interviewer can verify by running your code.
- **When you'd build something like this at a job:** any internal tool, data source or workflow you want multiple AI apps or agents to share, without a separate hand-written integration for each.
- **How it's built:** a `MCPServer` exposing 2 Tools (a write action; a read query), 1 Resource (a read-only listing), and 1 Prompt (a reusable template) — all backed by a real SQLite database.

Problems you'll likely run into, and how to fix them:

Problem	Fix
The Inspector connects, but `add_note` doesn't show up	Check the function is decorated with `@mcp.tool()`, and that `mcp.run(transport="stdio")` is the last line that runs — a server that never calls `mcp.run()` never listens
`add_note("")` silently creates a blank note	Validate `text.strip()` before touching the database, raise `EmptyNoteError`, and **catch it** to return a clear string — Doc01's "fail loudly" applied to a tool's content
A rejected note shows the client only "Error executing tool add_note"	Don't just raise — catch `EmptyNoteError` and `return f"Error: {e}"`, so the client sees the real reason

Problem	Fix
Notes disappear on restart	You're still on Step 1's in-memory list. Step 2 replaces it with SQLite
`search_notes("Dentist")` finds nothing	The search is case-sensitive — use `LOWER(text) LIKE ?` with `query.lower()`
The client hangs and never replies	Something wrote to **stdout** (a stray `print()`) — on stdio, stdout is the protocol channel. Route all logging to stderr via `get_logger`, never `print()`
You can't decide Tool vs. Resource	Does calling it *change* anything, or cost real computation? Yes → Tool. Only handing back data to read → Resource

Built During These Documents

01_python_foundations → 06_tools_function_calling (including its MCP sections)

Plan Before You Code

See 15_five_projects_index: write down Problem → Requirements → Architecture → Components → Data Flow → Implementation Plan → Coding Tasks. Deciding up front which of the 4 things (2 Tools, 1 Resource, 1 Prompt) each piece of behavior belongs to is exactly what this process is for.

How To Build This — Step by Step

The project grows step by step, and **after every step it runs**: Step 1 proves the connection, Step 2 makes the data real, Step 3 adds the Resource and the Prompt, Step 4 builds the client. Build it yourself first; use the hints if you're stuck, and the solution only to compare.

Every command runs from the **repo root** with the venv active.

How testing works in this project

Story: MCPForge's promise is "any client can connect, discover, and use what the server offers — and bad input is rejected cleanly." A casual `mcp dev` click can't confirm all of that, so every step adds the tests for what it builds. **If not:** a search bug would look like a model mistake, and a silently-stored blank note would only show up as junk data later.

- **Two kinds of test:**
  - `tests/unit/` — code that needs no server: the settings (`test_config.py`) and the SQLite helpers (`test_db.py`). No process to launch, under a second. **When:** after any

change to `config.py`, `db.py` or `exceptions.py`.

- `tests/integration/test_server.py` — the server, driven by a **real MCP client** over stdio. An MCP server can only be exercised through a client, so this test *is* a tiny client. **When:** at the end of every step, and after changing `server.py`.

- **One pattern for every case** — so the output never needs judging:

```
def test_something() -> None:
    result = "OK"
    if <it came out wrong>:
        result = "WRONG"
    print(f"case 1 (what it checks) -> {result}")
```

(The integration test uses a `show(case, passed)` helper that prints the same `OK` / `WRONG` line, since its cases are inside one async run.)

- **An error you expect** (like `MissingConfigError`): start with `result = "WRONG"`, set `result = "OK"` inside `except TheError:`.
- **Plain scripts, not pytest** — this project only uses Docs 01 and 06, which teach plain `print()` tests (the same style as Project 1). Run each with `python -m`.
- **Never `print()` in server code** — on stdio, stdout is the protocol channel. All logging goes to stderr via `get_logger`; a stray `print()` in `server.py` or anything it imports breaks the connection.
- **Got a `WRONG`?** Read that one case, then the code it calls. Fix the code — change the test only if the test checks the wrong thing.

Step 1 — A Minimal Server With One Tool

Project: MCPForge-Local-Notes-And-Tasks-Server — Step 1 of 4

- **Where you are now:** Setup is done — nothing runs yet.
- **What's new in this step:** the shared files (config, errors, stderr logger), `server.py` with one `@mcp.tool()` (`add_note`) backed by an in-memory list, and the first tests.
- **What stays the same:** — (first step).
- **Why:** every later layer is worthless if the protocol handshake doesn't work. Prove it alone — no database, no second tool.
- **How:** `MCPServer("notes-server")` → `@mcp.tool() add_note` → `mcp.run(transport="stdio")`. Test it with the Inspector, or with a tiny client in `test_server.py`.
- **Working after this step:** `mcp dev server.py` opens the Inspector and `add_note` responds; both test files print `OK`.

Read first: Doc06 — Core Concepts (what MCP is; servers and clients), Doc01 Build Task.

Stuck on this step? Hint 1 · Hint 2 · Show me the solution

What to do:

1. Bring in `src/config.py`, `src/exceptions.py` (`MissingConfigError`), and `src/utils/logger.py` — the logger on **`sys.stderr`**.
2. Write `server.py`: `MCPServer("notes-server")`, one `@mcp.tool()` `add_note(text: str) -> str` appending to an in-memory list, and `mcp.run(transport="stdio")` last.
3. Run `mcp dev server.py` and call `add_note` by hand, twice — both succeed; a restart resets the count (in-memory on purpose).
4. Write `tests/unit/test_config.py` and `tests/integration/test_server.py` (case 1), and run both with `python -m`.

The root files — `.gitignore`, `.env`, `.env.example`, `requirements.txt`, `LICENSE` — come from Setup; this step adds the files below.

Your files after Step 1:

```
<repo folder>/
├── src/
│   ├── utils/
│   │   ├── __init__.py     (new)
│   │   └── logger.py       (new) get_logger() -> stderr
│   ├── __init__.py
│   ├── config.py           (new)
│   └── exceptions.py       (new) MissingConfigError
├── tests/
│   ├── integration/
│   │   └── test_server.py  (new) case 1
│   └── unit/
│       └── test_config.py  (new) no server needed
└── server.py               (new) MCPServer, one tool, in-memory
```

Why these files (what and why):

- `src/utils/logger.py`
  - **What:** `get_logger(name)` with a handler on `sys.stderr`.
  - **Why:** stdout is the protocol channel; logging must go to stderr.
- `src/config.py`, `src/exceptions.py`
  - **What:** Doc01's config loader and `MissingConfigError`.
  - **Why:** one place for settings; a named error to catch.
- `server.py`
  - **What:** `MCPServer("notes-server")`, `add_note` as a tool, `mcp.run(transport="stdio")`.

    - **Why:** the smallest real server — proves the handshake before any real logic.
- `tests/unit/test_config.py`, `tests/integration/test_server.py`
    - **What:** the settings checks (no server) and the first client-driven check (tool listed + callable).
    - **Why:** prove the foundation before anything builds on it.

Tests for this step — write them yourself (the solution has the code, to compare):

- **cases 1-3 (settings)** — `tests/unit/test_config.py`
    - **Check:** `require_env()` raises `MissingConfigError` for a missing and an empty made-up setting; `load_config()` returns a `Config` with the default db path.
    - **Why:** the settings code must work with no server to launch.
    - **When:** after changing `config.py` or `exceptions.py`.
    - **If not:** a broken config would surface deep inside the server at startup.
- **case 1 (tools listed)** — `tests/integration/test_server.py`
    - **Check:** a client connects, `list_tools()` includes `add_note`, and `call_tool("add_note", ...)` returns a confirmation.
    - **Why:** proves the protocol handshake and the one tool actually work end to end.
    - **When:** at the end of the step, and after changing `server.py`.
    - **If not:** a server that never calls `mcp.run()` would look fine in code and never listen.

Step 2 — A Real SQLite-Backed Store, and the Search Tool

Project: MCPForge-Local-Notes-And-Tasks-Server — Step 2 of 4

- **Where you are now:** after Step 1 the server works, but forgets everything on restart and accepts any input.
- **What's new in this step:** `src/db.py` (real SQLite), `EmptyNoteError`, input validation on `add_note` (catch-and-return a clear string), and a second tool, `search_notes`. Tests: `test_db.py`, and search + empty-rejection cases in `test_server.py`.
- **What stays the same:** the server still exposes only Tools — the Resource and Prompt are Step 3.
- **Why:** a server that forgets everything isn't something a client would rely on, and a tool that stores a blank note is the "fail loudly, not silently" bug Doc01 and Doc06 exist to prevent.
- **How:** `init_db` / `insert_note` / `find_notes` in `db.py`; `add_note` strips and validates, raising `EmptyNoteError` and **catching it** to return `f"Error: {e}"` (the SDK only hands a raised error back as a generic message).
- **Working after this step:** notes survive a full restart, and empty text is rejected with a clear message.

Read first: Doc06 — Core Concepts (checking arguments; getting a tool's failure back correctly), Doc01 — Core Concepts.

Stuck on this step? Hint 1 · Hint 2 · Show me the solution

What to do:

1. Write `src/db.py`: `init_db(path)`, `insert_note(path, text)` (returns `lastrowid`), `find_notes(path, query)` (case-insensitive, `LOWER(text) LIKE ?`).
2. Add `EmptyNoteError` to `src/exceptions.py`.
3. Change `server.py`: `add_note` writes to the db and validates — raise `EmptyNoteError` on empty text, **catch it**, return `f"Error: {e}"`. Add `search_notes(query: str) -> list`.
4. Write `tests/unit/test_db.py` (insert, case-insensitive, no-match), add search + empty-rejection cases to `test_server.py`, and prove persistence by hand (restart the server).

Your files after Step 2:

```
<repo folder>/
├── src/
│   ├── db.py               (new) init_db/insert_note/find_notes
│   └── exceptions.py       (changed) + EmptyNoteError
├── tests/
│   ├── integration/
│   │   └── test_server.py  (changed) + search, empty-rejection
│   └── unit/
│       └── test_db.py      (new) no server needed
└── server.py               (changed) db-backed, validated, + search_notes
```

Why these files (what and why):

- `src/db.py`
  - **What:** all the SQL behind `init_db`, `insert_note`, `find_notes`.
  - **Why:** real persistence, testable with no server; a case-insensitive search.
- `src/exceptions.py`
  - **What:** adds `EmptyNoteError`.
  - **Why:** names the one failure `add_note` guards against.
- `server.py`
  - **What:** `add_note` now db-backed and validated (catch-and-return); `search_notes` is new.
  - **Why:** notes survive a restart; a rejected note shows the client a clear reason.
- `tests/unit/test_db.py`, `test_server.py`
  - **What:** the SQL helpers alone, plus client-driven search and rejection cases.
  - **Why:** a search bug is caught for free; the rejection is proven from the client's side.

Tests for this step — write them yourself:

- **cases 1-3 (db helpers)** — `tests/unit/test_db.py`

    - **Check:** `insert_note` returns id 1; `find_notes("DENTIST")` matches a `"dentist"` note; a no-match returns `[]`.
    - **Why:** proves persistence and case-insensitive search with no server.
    - **When:** after changing `db.py`.
    - **If not:** a case-sensitive search would look like a model mistake.
- **case 3 (search, case-insensitive)** — `tests/integration/test_server.py`
    - **Check:** `search_notes("MILK")` finds a `"Buy milk"` note through the client.
    - **Why:** proves the tool works over the protocol, not just in `db.py`.
    - **When:** at the end of the step.
    - **If not:** a wiring bug between the tool and `db.py` would go unnoticed.
- **case 4 (empty note rejected)** — `tests/integration/test_server.py`
    - **Check:** `add_note(" ")` returns a result whose text starts with `"Error"`.
    - **Why:** bad input must be rejected with a clear, client-readable message.
    - **When:** after changing the validation.
    - **If not:** blank notes would pile up, or the client would see a useless generic error.

Step 3 — Add the Resource and the Prompt

Project: MCPForge-Local-Notes-And-Tasks-Server — Step 3 of 4

- **Where you are now:** after Step 2 the server exposes two Tools, both backed by SQLite.
- **What's new in this step:** `list_all_notes` in `db.py`, a `@mcp.resource("notes://all")`, and a `@mcp.prompt()` `summarize_notes`. Tests: a `list_all_notes` case, and Resource + Prompt cases.
- **What stays the same:** `add_note` and `search_notes` don't change — they stay Tools, on purpose.
- **Why:** the code is small; the point is choosing the right MCP building block. A read-only listing is a Resource; a reusable template is a Prompt — not a third and fourth Tool.
- **How:** the Resource takes no arguments and returns a formatted string (with `"No notes yet."` when empty); the Prompt returns the *text* of a prompt, filled with the two dates, pointing at `notes://all`.
- **Working after this step:** the server exposes 2 Tools, 1 Resource, 1 Prompt.

Read first: Doc06 — Core Concepts (the 3 building blocks: Tools, Resources, Prompts).

Stuck on this step? Hint 1 · Hint 2 · Show me the solution

What to do:

1. Add `list_all_notes(path)` to `src/db.py` — a plain `SELECT id, text, created_at ... ORDER BY id`.

2. In `server.py`, add `@mcp.resource("notes://all")` (no arguments, returns a read-only listing, `"No notes yet."` when empty) and `@mcp.prompt()` `summarize_notes(start_date, end_date)` (returns a template string, never reads the db or calls a model).
3. Test both with the Inspector, then add a `list_all_notes` unit case and Resource + Prompt integration cases.

Your files after Step 3:

```
<repo folder>/
├── src/
│   └── db.py               (changed) + list_all_notes
├── tests/
│   ├── integration/
│   │   └── test_server.py  (changed) + resource, prompt
│   └── unit/
│       └── test_db.py      (changed) + list_all_notes case
└── server.py               (changed) + the Resource and the Prompt
```

Why these files (what and why):

- `src/db.py`
  - **What:** adds `list_all_notes(path)`.
  - **Why:** the Resource formats these rows; the query stays in `db.py`.
- `server.py`
  - **What:** adds the `notes://all` Resource and the `summarize_notes` Prompt.
  - **Why:** a read-only listing is a Resource; a reusable template is a Prompt — the distinction this step teaches.
- `tests/unit/test_db.py`, `test_server.py`
  - **What:** a `list_all_notes` case, plus client-driven Resource and Prompt cases.
  - **Why:** proves all three building blocks are reachable over the protocol.

Tests for this step — write them yourself:

- **case 4 (list all notes)** — `tests/unit/test_db.py`
  - **Check:** after two inserts, `list_all_notes` returns two rows, oldest first.
  - **Why:** the Resource depends on this order and shape.
  - **When:** after changing `list_all_notes`.
  - **If not:** the Resource could list notes in a surprising order.
- **case 5 (resource lists notes)** — `tests/integration/test_server.py`
  - **Check:** `read_resource("notes://all")` text contains a note you added.
  - **Why:** proves the Resource is reachable and reads real data.

    - **When:** at the end of the step.
    - **If not:** a client couldn't read your notes without a Tool call.
- **case 6 (prompt is a template)** — `tests/integration/test_server.py`
    - **Check:** `get_prompt("summarize_notes", {dates})` text contains a date you passed in.
    - **Why:** proves the Prompt is a filled-in template, not a summary or a db read.
    - **When:** at the end of the step.
    - **If not:** a Prompt that quietly read the db would blur the whole distinction this step teaches.

Step 4 — A Real Client That Connects and Uses All Three

Project: MCPForge-Local-Notes-And-Tasks-Server — Step 4 of 4

- **Where you are now:** after Step 3 the server exposes everything — but only the Inspector and the tests have used it.
- **What's new in this step:** a standalone `client.py` that launches the server, discovers what it offers, and uses all three building blocks in one run.
- **What stays the same:** `server.py` and everything in `src/` — this step proves the server from the outside, never touching its internals.
- **Why:** a server nobody has connected to isn't proven to work. A real client script proves the exact code path a future agent would use.
- **How:** `StdioServerParameters` + `stdio_client` + `ClientSession`, then `list_tools/resources/prompts` and `call_tool/read_resource/get_prompt`, wrapped in `try/except`.
- **Working after this step:** `python client.py` runs the full connect → discover → use loop in one go.

Read first: Doc06 — Core Concepts (servers and clients; why MCP matters for multi-agent systems).

Stuck on this step? Hint 1 · Hint 2 · Show me the solution

What to do:

1. Write `client.py`: `StdioServerParameters(command="python", args=["server.py"])`, `stdio_client(...)`, a `ClientSession`, and `await session.initialize()`.
2. Call `list_tools()`, `list_resources()`, `list_prompts()` and print each (build the name lists with plain for-loops, not a comprehension).
3. Use each one: `call_tool("add_note", ...)`, `call_tool("search_notes", ...)`, `read_resource("notes://all")`, `get_prompt("summarize_notes", {dates})`. Then call `add_note(" ")` and confirm the result text starts with `"Error"`.
4. Wrap it in `try/except` so a missing `server.py` prints one clear line. Run it; prove the failure path by renaming `server.py` briefly.

Your files after Step 4 (final): the full tree is in Project Structure. What changed:

```
<repo folder>/
└── client.py               (new) standalone MCP client: discover + use
```

Why these files (what and why):

- `client.py`
  - **What:** launches the server, discovers its tools/resources/prompts, and uses all three, with a clear failure path.
  - **Why:** proves the full loop from the outside — the exact shape a real agent's MCP integration uses.

Tests for this step — write them yourself:

- **run `client.py`** — the project's end-to-end proof
  - **Check:** one run prints the discovered tools/resources/prompts and a readable result for each of the four calls, plus the empty-note rejection.
  - **Why:** the integration test proves the protocol surface; `client.py` proves the real client code path a future agent uses.
  - **When:** after writing `client.py`, and whenever `server.py` changes.
  - **If not:** you'd have a server nobody has actually connected to with real client code.

Final Deliverable: MCPForge-Local-Notes-And-Tasks-Server — a real MCP server exposing 2 Tools (`add_note` , `search_notes`), 1 Resource (`notes://all`), and 1 Prompt (`summarize_notes`), backed by SQLite, plus a standalone `client.py` that proves the full connect → discover → use loop works end to end.

Why each is a Tool / Resource / Prompt (said plainly): `add_note` and `search_notes` *do* something — write, or run a query — so they're Tools. `notes://all` only hands back data to read — so it's a Resource. `summarize_notes` is a reusable, parameterized template — so it's a Prompt. Getting that split right, not the SQLite code, is what this project proves.

Run, Test and Configure the Final Project

The same commands and settings as the repo's README.md — once Step 4 is done, this is how the finished project is run, tested and configured.

Configuration

Variable	Required	Default	Meaning
`LOG_LEVEL`	no	`INFO`	`DEBUG` , `INFO` , `WARNING` or `ERROR`

Variable	Required	Default	Meaning
`NOTES_DB_PATH`	no	`notes.db`	Where the SQLite notes file lives

Setting	Where	Default
Server name	`server.py` → `MCPServer("notes-server")`	`notes-server`
Database file	`.env` → `NOTES_DB_PATH`	`notes.db`

Usage

Run from the repo root:

```
mcp dev server.py       # the MCP Inspector, to call tools by hand
python client.py        # the standalone client: discover + use everything
```

To connect from Claude Desktop or another MCP client, point it at `python server.py` from this folder.

What the server exposes	Kind	What it does
`add_note(text)`	Tool	Stores a note; rejects empty text with a clear error
`search_notes(query)`	Tool	Case-insensitive substring search over stored notes
`notes://all`	Resource	A read-only listing of every note, oldest first
`summarize_notes(start_date, end_date)`	Prompt	A filled-in template asking a model to summarize notes in a date range

Running the Tests

Run from the repo root:

```
# unit tests — no server, no network
python -m tests.unit.test_config
python -m tests.unit.test_db

# integration test — launches server.py via a real MCP client
python -m tests.integration.test_server
```

The unit tests check the settings and the SQLite helpers. The integration test launches the server and, through a real client, confirms the tools are listed, `add_note` and `search_notes` work, an

empty note is rejected, the Resource lists notes, and the Prompt is a filled-in template.

Checklist Before You Call This Done

- [ ] The repo folder name matches the GitHub repo name exactly
- [ ] Code lives in `src/`, with `server.py` and `client.py` at the repo root
- [ ] Step 1's minimal server starts, and a client (Inspector or your own) calls `add_note`
- [ ] `add_note` and `search_notes` are backed by SQLite — notes survive a full restart
- [ ] Empty or whitespace-only text is rejected with a clear error the client can read, never stored
- [ ] `notes://all` is a Resource, not a third Tool — you can explain why
- [ ] `summarize_notes` is a Prompt, parameterized by a date range, not a hardcoded string
- [ ] `client.py` connects, lists tools/resources/prompts, calls a tool, reads the resource, and uses the prompt — all in one run
- [ ] Nothing in the server (or anything it imports) ever calls `print()` — logging goes to stderr
- [ ] `tests/unit/` pass with no server; `tests/integration/` passes by launching the server
- [ ] You can explain, for each of the 4 things the server exposes, why it's a Tool vs. a Resource vs. a Prompt

Publish It to GitHub

1. Copy README.md from this folder into your repo folder, and put your own name in its License line.
2. Check nothing secret is staged, then commit:

```
git status                      # .env and .venv/ must NOT appear
git add .
git commit -m "MCPForge: a local notes-and-tasks MCP server and client"
```

3. On GitHub, create a new repo named exactly `MCPForge-Local-Notes-And-Tasks-Server` (no README — you have one). In the repo's **Description** box, paste the **Short description** line from the top of README.md. Then:

```
git branch -M main
git remote add origin <your-repo-url>
git push -u origin main
```

Status

Not started. Track your progress in ../PROGRESS.md.

Stuck? Ask for **Hint 1** or **Hint 2** about the exact part you're stuck on. Only ask for the full code if you say **"Show me the solution."**